



Aplicación de Algoritmos de Detección de Anomalías

0) Preparación — Generación de los 3 datasets sintéticos en memoria

Objetivo del bloque: crear **todos los datasets en memoria** (sin leer ni escribir archivos) y dejar listas las utilidades de evaluación. Fijamos semillas para **reproducibilidad**, definimos la **verdad terreno** (anomaly_true: -1 = anomalía, 1 = normal) y funciones de **métricas** (precision/recall/F1) para reutilizar en toda la hoja.

```
1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4
5  from sklearn.metrics import precision_score, recall_score,
f1_score
6  from sklearn.ensemble import IsolationForest
7  from sklearn.neighbors import LocalOutlierFactor
8  from sklearn.cluster import DBSCAN
9  from sklearn.preprocessing import StandardScaler
10 from sklearn.svm import OneClassSVM
11 from sklearn.datasets import make_blobs
12
13 # -----
14 # 0.A) Dataset UNIVARIANTE
15 # -----
16 rng = np.random.default_rng(42)
17 n_norm, n_out = 500, 15
18 normal = rng.normal(50, 5, n_norm)
19 outliers = rng.normal(90, 3, n_out)
20 serie = np.concatenate([normal, outliers])
21 labels_uni = np.array([1]*n_norm + [-1]*n_out)
22 perm = rng.permutation(len(serie))
23 serie = serie[perm]
24 labels_uni = labels_uni[perm]
25 uni = pd.DataFrame({'valor': serie, 'anomaly_true':
labels_uni})
26
```

```

27 # -----
28 # 0.B) Dataset MULTIVARIANTE (2D)
29 # -----
30 X, _ = make_blobs(n_samples=800, centers=[(-2,-2),(3,3)],
31                  cluster_std=[0.8,1.0], random_state=7)
32 rng2 = np.random.default_rng(7)
33 X_out_g = rng2.uniform(-10, 10, size=(12, 2))
34 X_out_l = np.c_[rng2.normal(3, 0.3, 12), rng2.normal(-3, 0.3,
35 12)]
36 X_all = np.vstack([X, X_out_g, X_out_l])
37 labels_multi = np.hstack([np.ones(len(X)), -np.ones(24)])
38 multi = pd.DataFrame(X_all, columns=['x1', 'x2'])
39 multi['anomaly_true'] = labels_multi
40 # -----
41 # 0.C) Dataset SERIE TEMPORAL
42 # -----
43 n = 600
44 idx = pd.date_range('2022-01-01', periods=n, freq='H')
45 trend = np.linspace(0, 10, n)
46 season = 5 * np.sin(2*np.pi*np.arange(n)/24)
47 noise = rng.normal(0, 0.8, n)
48 base = 20 + trend + season + noise
49 s = base.copy()
50 spikes = rng.choice(n, size=10, replace=False)
51 s[spikes] += rng.normal(12, 3, size=10)
52 start = 300
53 window_len = 24
54 noisy_idx = np.arange(start, start+window_len)
55 s[noisy_idx] += rng.normal(0, 4, window_len)
56 labels_ts = np.ones(n, dtype=int)
57 labels_ts[spikes] = -1
58 labels_ts[noisy_idx] = -1
59 ts = pd.DataFrame({'timestamp': idx, 'valor': s, 'base': base,
60 'anomaly_true': labels_ts})
61 # -----
62 # 0.D) Métricas de evaluación reutilizables
63 # -----
64 def resumen_metricas(y_true, y_pred, nombre_modelo):
65     y_true = np.asarray(y_true)

```

```

66     y_pred = np.asarray(y_pred)
67     prec = precision_score(y_true == -1, y_pred == -1,
zero_division=0)
68     rec = recall_score(y_true == -1, y_pred == -1,
zero_division=0)
69     f1 = f1_score(y_true == -1, y_pred == -1,
zero_division=0)
70     return {'modelo': nombre_modelo, 'precision': prec,
'recall': rec, 'f1': f1}
71

```

1) Caso univariante

Objetivo del bloque: comparar enfoques **univariantes** ante *outliers* globales.

Z-Score: asume distribución ~normal; marca puntos a $> k$ sigmas ($k \approx 3$).

IQR: robusto a colas/asimetría; usa límites $Q1 - 1.5 \cdot IQR$ y $Q3 + 1.5 \cdot IQR$.

Isolation Forest (IF): aísla observaciones inusuales en árboles aleatorios; no requiere supuestos de distribución.

Qué observar: detecciones, concordancia con *anomaly_true* y equilibrio **precisión–recall**.

1.1 Visualización

```

1  plt.plot(uni.index, uni['valor'], '.-', alpha=0.7)
2  plt.title('Univariante – valores con outliers')
3  plt.xlabel('Índice'); plt.ylabel('Valor')
4  plt.show()
5

```

1.2 Z-Score

```

1  mu = uni['valor'].mean()
2  sigma = uni['valor'].std(ddof=0) if uni['valor'].std(ddof=0) >
0 else 1
3  uni['z'] = (uni['valor'] - mu) / sigma
4  uni['pred_z'] = np.where(np.abs(uni['z'])>3, -1, 1)
5  res_z = resumen_metricas(uni['anomaly_true'], uni['pred_z'],
'Z-Score')
6  res_z
7

```

1.3 IQR

```
1 q1, q3 = uni['valor'].quantile(0.25),  
  uni['valor'].quantile(0.75)  
2 iqr = q3 - q1  
3 low, high = q1 - 1.5*iqr, q3 + 1.5*iqr  
4 uni['pred_iqr'] =  
  np.where((uni['valor']<low)|(uni['valor']>high), -1, 1)  
5 res_iqr = resumen_metricas(uni['anomaly_true'],  
  uni['pred_iqr'], 'IQR')  
6 res_iqr  
7
```

1.4 Isolation Forest

```
1 iso = IsolationForest(contamination=0.03, random_state=42)  
2 uni['pred_if'] = iso.fit_predict(uni[['valor']])  
3 res_if = resumen_metricas(uni['anomaly_true'], uni['pred_if'],  
  'Isolation Forest')  
4 res_if  
5
```

1.5 Comparativa

```
1 res_uni = pd.DataFrame([res_z, res_iqr, res_if])  
2 res_uni  
3
```

2) Caso multivariante (2D)

Objetivo del bloque: detectar anomalías cuando **importa la relación entre variables**.

LOF: compara densidad local vs. vecinos → capta *outliers locales*.

DBSCAN (con *scaling*): clustering por densidad; marca **ruido** (cluster -1) → útil para *outliers colectivos*; sensible a *eps* y *min_samples*.

IF (2D): generalista, robusto ante *outliers* globales.

OCSVM: aprende frontera de normalidad; sensible a *scaling* y parámetros (*nu*, *gamma*).

Qué observar: diferencias entre **local** (LOF) y **ruido** (DBSCAN), y trade-offs de IF/OCSVM.

2.1 Visualización inicial

```
1 plt.figure(figsize=(5,5))
```

```

2 plt.scatter(multi['x1'], multi['x2'], c=multi['anomaly_true'],
s=12)
3 plt.title('Multivariante – verdad terreno (-1 = rojo, 1 =
azul)')
4 plt.show()
5

```

2.2 LOF

```

1 lof = LocalOutlierFactor(n_neighbors=20, contamination=0.03)
2 multi['pred_lof'] = lof.fit_predict(multi[['x1','x2']])
3 res_lof = resumen_metricas(multi['anomaly_true'],
multi['pred_lof'], 'LOF')
4 res_lof
5 plt.figure(figsize=(5,5))
6 plt.scatter(multi['x1'], multi['x2'], c=multi['pred_lof'],
s=12)
7 plt.title('LOF – predicción (-1 = rojo, 1 = azul)')
8 plt.show()
9

```

2.3 DBSCAN (con StandardScaler)

```

1 Xm = StandardScaler().fit_transform(multi[['x1','x2']])
2 db = DBSCAN(eps=0.7, min_samples=10)
3 clusters = db.fit_predict(Xm)
4 multi['pred_dbscan'] = np.where(clusters== -1, -1, 1)
5 res_db = resumen_metricas(multi['anomaly_true'],
multi['pred_dbscan'], 'DBSCAN')
6 res_db
7 plt.figure(figsize=(5,5))
8 plt.scatter(multi['x1'], multi['x2'], c=multi['pred_dbscan'],
s=12)
9 plt.title('DBSCAN – predicción (-1 = rojo, 1 = azul)')
10 plt.show()
11

```

2.4 Isolation Forest (2D)

```

1 iso_m = IsolationForest(contamination=0.03, random_state=42)
2 multi['pred_if'] = iso_m.fit_predict(multi[['x1','x2']])
3 res_ifm = resumen_metricas(multi['anomaly_true'],
multi['pred_if'], 'Isolation Forest (2D)')
4 res_ifm

```

```

5 plt.figure(figsize=(5,5))
6 plt.scatter(multi['x1'], multi['x2'], c=multi['pred_if'], s=12)
7 plt.title('Isolation Forest – predicción (-1 = rojo, 1 =
azul)')
8 plt.show()
9

```

2.5 One-Class SVM (RBF)

```

1 Xm = StandardScaler().fit_transform(multi[['x1','x2']])
2 oc = OneClassSVM(kernel='rbf', gamma='scale', nu=0.03)
3 multi['pred_oc'] = oc.fit_predict(Xm)
4 res_oc = resumen_metricas(multi['anomaly_true'],
multi['pred_oc'], 'OCSVM')
5 res_oc
6 plt.figure(figsize=(5,5))
7 plt.scatter(multi['x1'], multi['x2'], c=multi['pred_oc'], s=12)
8 plt.title('OCSVM – predicción (-1 = rojo, 1 = azul)')
9 plt.show()
10

```

2.6 Comparativa

```

1 res_multi = pd.DataFrame([res_lof, res_db, res_ifm, res_oc])
2 res_multi
3

```

3) Caso de serie temporal

Objetivo del bloque: tratar anomalías **contextuales** y **colectivas** en una serie con **tendencia** + **estacionalidad**.

Z-Score (24h): normaliza respecto al contexto local; detecta *spikes* altos.

****IF con **features:** añadir z24 y std24 aporta contexto; capta *spikes* y segmentos.

Regla colectiva (std en ventana): identifica **bloques ruidosos** aunque cada punto no sea extremo.

Qué observar: sensibilidad a *spikes* vs. segmentos, e impacto de *features* de ventana.

3.1 Visualización

```

1 plt.plot(ts['timestamp'], ts['valor'], '-', alpha=0.8)
2 plt.title('Serie temporal – valor')
3 plt.show()

```

3.2 Z-Score en ventana (24h)

```

1  roll_mean = ts['valor'].rolling(24, center=True,
min_periods=12).mean()
2  roll_std  = ts['valor'].rolling(24, center=True,
min_periods=12).std()
3  z24 = (ts['valor'] - roll_mean) / roll_std.replace(0, np.nan)
4  z24 = z24.fillna(0)
5  pred_z24 = np.where(np.abs(z24)>3, -1, 1)
6  res_z24 = resumen_metricas(ts['anomaly_true'], pred_z24,
'Temporal - Z24')
7  res_z24
8

```

plt.plot(ts['timestamp'], ts['valor'], '-', alpha=0.7) plt.scatter(ts['timestamp'][pred_z24==-1], ts['valor'][pred_z24==-1], c='red', s=15, label='Anomalía') plt.title('Serie temporal — Z-Score (24h)') plt.legend(); plt.show()

1

3.3 Isolation Forest con features (valor, z24, std24)

```

1  ts_feat = pd.DataFrame({'valor': ts['valor'], 'z24': z24,
'std24': roll_std.fillna(0)})
2  iso_t = IsolationForest(contamination=0.04, random_state=42)
3  pred_if_t = iso_t.fit_predict(ts_feat)
4  res_if_t = resumen_metricas(ts['anomaly_true'], pred_if_t,
'Temporal - IF(features)')
5  res_if_t
6

```

plt.plot(ts['timestamp'], ts['valor'], '-', alpha=0.7) plt.scatter(ts['timestamp'][pred_if_t==-1], ts['valor'][pred_if_t==-1], c='red', s=15, label='Anomalía') plt.title('Serie temporal — Isolation Forest (features)') plt.legend(); plt.show()

1

3.4 Regla colectiva (varianza en ventana)

```

1  rs = roll_std.dropna()
2  q1s, q3s = rs.quantile(0.25), rs.quantile(0.75)
3  iqr = q3s - q1s
4  thr = q3s + 1.5*iqr
5  pred_coll = np.where(roll_std > thr, -1, 1)

```

```

6 res_coll = resumen_metricas(ts['anomaly_true'], pred_coll,
  'Temporal - Colectiva(std)')
7 res_coll
8

```

```

plt.plot(ts['timestamp'], ts['valor'], '-', alpha=0.7) plt.scatter(ts['timestamp'][pred_coll==1],
ts['valor'][pred_coll==1], c='red', s=15, label='Anomalía') plt.title('Serie temporal — Regla
colectiva (std en ventana)') plt.legend(); plt.show()

```

```
1
```

3.5 Comparativa

```

1 res_ts = pd.DataFrame([res_z24, res_if_t, res_coll])
2 res_ts
3

```

4) Comparativa global

Objetivo del bloque: consolidar resultados en una **tabla** y discutir **trade-offs** por dominio (univariante/2D/temporal).

Precision alta suele implicar **recall bajo** (DBSCAN conservador).

IF ofrece equilibrio pero requiere *tuning* de contamination.

No hay ganador universal: el contexto manda.

```

1 resumen = (pd.concat([
2     res_uni.assign(dataset='Univariante'),
3     res_multi.assign(dataset='Multivariante'),
4     res_ts.assign(dataset='Temporal')
5 ])[['dataset', 'modelo', 'precision', 'recall', 'f1']].reset_index(
  drop=True))
6 resumen
7

```

5) Extensiones y tuning

Objetivo del bloque: experimentar con **hiperparámetros** para mejorar F1 según tu problema.

LOF: variar `n_neighbors`. **DBSCAN:** cuadrícula sobre `eps` y `min_samples` (tras `StandardScaler`).

IF: ajustar contamination y max_samples. **OCSVM:** probar nu (fracción de outliers) y gamma (ancho del RBF).

```
1 # Barrido rápido de LOF en 2D
2 valores_k = [5, 10, 20, 50]
3 res_k = []
4 for k in valores_k:
5     lof_k = LocalOutlierFactor(n_neighbors=k,
6 contamination=0.03)
7     pred_k = lof_k.fit_predict(multi[['x1', 'x2']])
8     res_k.append(resumen_metricas(multi['anomaly_true'],
9 pred_k, f'LOF k={k}'))
10
```